

AnchorIQ 트러블슈팅 기록

선박/해운 공급망 리스크 플랫폼 — 데이터 파이프라인 디버깅

2026-03-31

Redis 포트 충돌로 Docker Redis 대신 로컬 Redis 연결: DBSIZE 0 → 17,301건 정상 조회

배경 및 문제정의

AIS 선박 위치 데이터를 Redis GEO에 저장한 뒤 지도에 표시하는 파이프라인에서, `docker exec infra-redis-1 redis-cli DBSIZE` 가 항상 0을 반환했다. Spring Boot 앱 내부 디버그 엔드포인트(`/api/debug/redis/dbsize`)로 확인하면 17,301건이 존재했다. 동일한 Redis를 바라보는데 결과가 다른 상황이었다.

원인은 호스트에 Homebrew Redis와 Docker Redis가 동시에 6379 포트를 점유하고 있었고, Spring Boot는 localhost:6379 → Homebrew Redis에, docker exec는 컨테이너 내부 Redis에 연결되어 서로 다른 인스턴스를 바라보고 있었다.

기술 선정 (대안 비교)

대안	장점	단점	선택
Homebrew Redis 중지 (<code>brew services stop redis</code>)	포트 해제 간단	다른 프로젝트 영향, 재부팅 시 재시작 가능	X
Docker Redis 포트를 6380으로 변경	충돌 완전 제거, 명시적 분리	설정 파일 변경 필요	O
Spring Boot에서 Docker 내부 IP 직접 지정	포트 변경 불필요	컨테이너 IP 유동적, 유지보수 어려움	X

분석

초기 판단

“Redis에 데이터가 안 들어갔다”고 판단. Kafka Consumer 로그를 확인하니 정상 consume + Redis 저장 성공 로그가 찍히고 있었다.

실제 원인

```
# 포트 점유 확인
$ lsof -i :6379
COMMAND  PID  USER  FD  TYPE  DEVICE  NODE  NAME
redis-ser 512  user   6u  IPv4  ...    TCP    *:6379 (LISTEN) #
           Homebrew
com.docke 891  user  98u  IPv6  ...    TCP    *:6379 (LISTEN) #
           Docker proxy
```

Homebrew Redis가 먼저 6379를 바인딩. Docker Desktop의 포트 매핑(-p 6379:6379)은 `SO_REUSEADDR` 로 같은 포트에 바인딩을 시도하지만, macOS에서 IPv4/IPv6 스택이 분리되어 각각 LISTEN 상태가 된다. `localhost` 해석 시 IPv4(127.0.0.1)가 우선 → Homebrew Redis에 연결.

CS 원리: TCP 포트 바인딩과 Docker Port Mapping

TCP 포트 바인딩: 커널은 (protocol, local_addr, local_port) 튜플로 소켓을 식별한다. `SO_REUSEADDR` 소켓 옵션이 설정되면 동일 포트에 여러 소켓이 바인딩 가능하고, 정확히 같은 주소 (예: 둘 다 0.0.0.0:6379)면 충돌한다. 하지만 하나가 IPv4(0.0.0.0), 다른 하나가 IPv6(::)이면 커널이 별도 소켓으로 허용한다.

Docker Port Mapping: `docker-proxy` 프로세스가 호스트 포트에서 LISTEN → 수신된 패킷을 iptables/userland proxy로 컨테이너 veth 인터페이스에 전달. macOS에서는 LinuxKit VM 내부 포워딩이므로 호스트 네이티브 소켓과 충돌 감지가 불완전하다.

Lettuce AUTH 동작: Spring Data Redis의 Lettuce 클라이언트는 연결 시 `AUTH password` 명령을 전송한다. 비밀번호가 설정되지 않은 Redis(Homebrew)에 AUTH를 보내면 `ERR Client sent AUTH, but no password is set` 응답이 오지만, **Lettuce는 이를 무시하고 연결을 유지한다.** Redis AUTH 프로토콜(RESP)에서 AUTH 실패는 연결을 끊지 않으며, 이후 명령은 정상 실행된다.

```
# Redis AUTH 프로토콜 (RESP)
Client: *2\r\n$4\r\nAUTH\r\n$8\r\npassword\r\n
Server: -ERR Client sent AUTH, but no password is set\r\n ← 에러지만 연결 유지
Client: *1\r\n$4\r\nPING\r\n
Server: +PONG\r\n ← 정상 동작
```

솔루션

Docker Redis 포트를 6380으로 변경하여 충돌을 제거.

```
# docker-compose.yml
services:
  redis:
    image: redis:7-alpine
    ports:
      - "6380:6379"      # 호스트 6380 → 컨테이너 6379
    command: redis-server --requirepass ${REDIS_PASSWORD}
```

```
# application-local.yml
spring:
  data:
    redis:
      host: localhost
      port: 6380          # Docker Redis 전용 포트
      password: ${REDIS_PASSWORD}
```

```
# 검증
$ docker exec infra-redis-1 redis-cli -a $REDIS_PASSWORD DBSIZE
(integer) 17301

$ redis-cli -p 6380 -a $REDIS_PASSWORD DBSIZE
(integer) 17301    # 동일 인스턴스 확인
```

결과 (Before / After)

지표	Before	After
<code>docker exec redis-cli DBSIZE</code>	0 (잘못된 인스턴스)	17,301
Spring Boot 연결 대상	Homebrew Redis (6379/IPv4)	Docker Redis (6380)
AIS 선박 지도 표시	0척	17,000+척
포트 충돌	IPv4/IPv6 이중 LISTEN	완전 분리
디버깅 소요 시간	2시간 (원인 불명)	즉시 확인 가능

Redis GEO TTL/Cleanup 레이스 컨디션: 선박 위치 0건 → 17,000+건 유지

배경 및 문제정의

AIS 수집 파이프라인이 Kafka → AisRedisConsumer → Redis GEO(vessels:positions)로 선박 위치를 저장하고, RedisGeoCleanupScheduler 가 오래된 데이터를 주기적으로 정리하는 구조였다. 데이터가 저장되자마자 사라지는 현상이 발생했다.

AisRedisConsumer 는 GEO 저장 후 vessels:timestamp:{mmsi} 에 TTL 30초를 설정. RedisGeoCleanupScheduler 는 1분마다 실행되어 timestamp 키가 만료된 MMSI를 GEO에서 ZREM 으로 삭제. **TTL(30초) < Cleanup 주기(60초)** 이므로, Cleanup 실행 시점에 거의 모든 timestamp가 만료 상태 → GEO 전체 삭제.

기술 선정 (대안 비교)

대안	장점	단점	선택
TTL 제거 (영구 보관)	삭제 문제 없음	메모리 무한 증가, 비활성 선박 잔류	X
TTL을 Cleanup 주기보다 길게 설정	간단, 레이스 제거	TTL/주기 비율 관리 필요	O
Cleanup에서 TTL 잔여 시간 확인 후 삭제	정밀한 제어	TTL 명령 N번 호출 → O(N) 부하	X
Redis Keyspace Notification으로 이벤트 기반 삭제	실시간 정리	설정 복잡, pub/sub 신뢰성 이슈	X

분석

초기 판단

“수집기가 데이터를 안 보내는 것 아닌가?” → Kafka Consumer 로그 확인 시 정상 consume. Redis MONITOR 명령으로 실시간 확인 시 GEOADD 직후 ZREM 이 따라오는 패턴 발견.

실제 원인: TTL과 Cleanup 주기의 타이밍 역전

```
Timeline:
T=0s   AisRedisConsumer: GEOADD vessels:positions ... mmsi_123
T=0s   AisRedisConsumer: SET vessels:timestamp:mmsi_123 EX 30
T=30s  Redis: DEL vessels:timestamp:mmsi_123 (TTL 만료, 자동 삭제)
```

```
T=60s Scheduler: SCAN vessels:timestamp:* → mmsi_123 없음 → ZREM vessels:positions mmsi_123
↑ 30초 전에 만료됐으므로 "비활성"으로 판단 → 삭제
```

CS 원리: TTL, Eviction, Redis GEO 내부 구조

TTL과 Eviction 전략: Redis는 키 만료를 두 가지 방식으로 처리한다. (1) **Lazy Expiration** - 클라이언트가 키에 접근할 때 만료 확인 후 삭제. (2) **Active Expiration** - 매 100ms마다 랜덤 20개 키를 샘플링하여 만료된 키 삭제. 만료된 키가 25% 이상이면 즉시 반복. 이 확률적 방식 때문에 TTL 만료 후 실제 삭제까지 수백ms 지연이 있을 수 있으나, `EXISTS` 검사 시에는 만료된 키를 즉시 없는 것으로 처리한다.

Redis GEO 내부 구조: `GEODADD` 는 내부적으로 Sorted Set(`ZADD`)으로 구현된다. 좌표 (longitude, latitude) 를 52비트 Geohash로 인코딩하여 score로 저장한다.

```
GEODADD vessels:positions 129.05 35.10 "mmsi_440123456"
→ 내부: ZADD vessels:positions 4065562751.0 "mmsi_440123456"
      ↑ geohash(129.05, 35.10)의 52비트 정수값
```

Geohash는 Z-order curve(Morton code)를 사용하여 2D 좌표를 1D 정수로 매핑한다. 인접한 좌표는 인접한 정수값을 가지므로 `GEORADIUS` 가 Sorted Set의 range query로 $O(\log(N)+M)$ 효율을 달성한다.

Race Condition: 이 문제는 전통적인 멀티스레드 레이스 컨디션과 다르다. 두 개의 독립된 타이머(TTL 만료, Scheduler 실행)가 **공유 자원(GEO 키)**에 대해 서로 다른 시점에 동작하면서 발생하는 **temporal race condition**이다. 단일 스레드 Redis에서도 클라이언트 측 스케줄링 타이밍 차이로 발생한다.

Stale Data vs Freshness 트레이드오프: TTL을 짧게 하면 freshness가 높지만 데이터 소실 위험 증가. TTL을 길게 하면 stale data가 남지만 가용성 증가. AIS 위치 데이터는 5분 이내면 "실시간"으로 간주되므로 TTL 5분이 적절한 균형점이다.

솔루션

TTL을 Cleanup 주기보다 충분히 길게 설정하여 레이스 윈도우를 제거.

```
// AisRedisConsumer.java - TTL 5분으로 변경
private static final Duration POSITION_TTL = Duration.ofMinutes(5);

private void saveToRedisGeo(AisPositionMessage message) {
    Point point = new Point(message.getLongitude(),
        message.getLatitude());
    redisTemplate.opsForGeo()
        .add("vessels:positions", point,
            String.valueOf(message.getMmsi()));

    // timestamp 키에 TTL 5분 설정 (cleanup 주기 6분보다 짧지만 충분한 여유)
    String timestampKey = "vessels:timestamp:" + message.getMmsi();
    redisTemplate.opsForValue()
```

```

        .set(timestampKey,
            String.valueOf(Instant.now().toEpochMilli()), POSITION_TTL);
    }

```

```

// RedisGeoCleanupScheduler.java - 주기 6분으로 변경
@Scheduled(fixedRate = 360_000) // 6분 (TTL 5분 + 1분 여유)
public void cleanupStalePositions() {
    Set<String> activeKeys =
        redisTemplate.keys("vessels:timestamp:*");
    Set<String> geoMembers = redisTemplate.opsForZSet()
        .range("vessels:positions", 0, -1);

    if (geoMembers == null) return;

    Set<String> activeMmsiSet = activeKeys.stream()
        .map(key -> key.replace("vessels:timestamp:", ""))
        .collect(Collectors.toSet());

    // timestamp가 만료된(=키가 없는) MMSI만 GEO에서 제거
    geoMembers.stream()
        .filter(mmsi -> !activeMmsiSet.contains(mmsi))
        .forEach(mmsi -> redisTemplate.opsForGeo()
            .remove("vessels:positions", mmsi));
}

```

핵심 공식: $TTL < \text{Cleanup 주기} < TTL + \text{수집 간격}$. 수집이 TTL 내에 갱신하면 timestamp가 리프레시되므로 활성 선박은 절대 삭제되지 않는다.

결과 (Before / After)

지표	Before	After
GEO TTL	30초	5분
Cleanup 주기	1분	6분
GEORADIUS 조회 결과	0척	17,000+척
비활성 선박 정리	전부 삭제 (과잉 정리)	5분 이상 미갱신 선박만 정리
데이터 freshness	N/A (데이터 없음)	5분 이내 실시간

WebSocket 바이너리 메시지 1003 Close 무한 루프: AIS 수신 0 → 초당 70+ 메시지

배경 및 문제정의

`AisStreamWebSocketClient` 가 `AISstream.io`에 연결하여 실시간 AIS 메시지를 수신하는 구조에서, 연결 성공 → 구독 메시지 전송 → 즉시 1003 Close → 재연결 → 반복하는 무한 루프가 발생했다. Kafka `ais-positions` 토픽에 메시지가 0건이었다.

Spring 로그에 `Standard WebSocket close: code=1003, reason=Binary messages not supported` 가 반복 출력.

기술 선정 (대안 비교)

대안	장점	단점	선택
<code>TextWebSocketHandler</code> 유지 + 서버에 텍스트 요청	코드 변경 최소	<code>AISstream.io</code> 가 바이너리 전송을 제어 불가	X
<code>BinaryWebSocketHandler</code> 로 변경	바이너리 처리 가능	텍스트 메시지(구독 응답 등) 처리 불가	X
<code>AbstractWebSocketHandler</code> 로 변경	텍스트+바이너리 모두 처리	두 핸들러 메서드 구현 필요	O
외부 WebSocket 라이브러리 사용	유연한 프레임 처리	Spring 생태계 이탈, 의존성 추가	X

분석

초기 판단

“서버가 연결을 거부하는 것 아닌가?” → 연결 자체는 성공(HTTP 101 Switching Protocols 확인). 구독 메시지 전송까지 성공. 그 직후 Close 프레임 수신.

실제 원인

`AisStreamWebSocketClient` 가 `TextWebSocketHandler` 를 상속하고 있었다. `AISstream.io` 서버는 데이터를 **바이너리 WebSocket 프레임(opcode 0x2)**으로 전송한다.

`TextWebSocketHandler` 는 `handleBinaryMessage` 를 오버라이드하여 `CloseStatus(1003)` Close 프레임을 전송한다.

```
// Spring TextWebSocketHandler 내부 구현
public class TextWebSocketHandler extends AbstractWebSocketHandler {
    @Override
```

```
protected void handleBinaryMessage(WebSocketSession session,
    BinaryMessage message) {
    try {
        session.close(CloseStatus.NOT_ACCEPTABLE); // 1003 전송
    } catch (IOException ex) {
        // ...
    }
}
```

CS 원리: WebSocket 프로토콜 (RFC 6455)

WebSocket 프레임 구조: WebSocket은 TCP 위에 프레임 기반 프로토콜을 정의한다. 각 프레임의 첫 바이트에 opcode가 포함된다.

- **opcode 0x1 (Text Frame):** 페이로드가 UTF-8 텍스트. 수신 측은 UTF-8 유효성을 검증해야 한다.
- **opcode 0x2 (Binary Frame):** 페이로드가 임의의 바이트 시퀀스. 인코딩 검증 없음.
- **opcode 0x8 (Close Frame):** 연결 종료. 페이로드 첫 2바이트가 상태 코드(빅엔디언).

Close Code 1003 (NOT_ACCEPTABLE): RFC 6455 Section 7.4.10에 정의. “수신한 데이터 타입을 처리할 수 없음”을 의미. 텍스트만 지원하는 엔드포인트가 바이너리를 수신하거나 그 반대일 때 사용한다.

Spring WebSocket 핸들러 계층:

```

AbstractWebSocketHandler
├─ handleMessage()      → 기본: 아무것도 안 함
├─ handleBinaryMessage() → 기본: 아무것도 안 함
└─ handlePongMessage()  → 기본: 아무것도 안 함

TextWebSocketHandler extends AbstractWebSocketHandler
└─ handleBinaryMessage() → 오버라이드: session.close(1003)

BinaryWebSocketHandler extends AbstractWebSocketHandler
└─ handleMessage()      → 오버라이드: session.close(1003)

```


TextWebSocketHandler 는 “텍스트만 허용”이라는 의미를 핸들러 레벨에서 강제하는 설계다. 바이너리 수신 시 프로토콜 표준에 따라 1003을 보내는 것은 올바른 동작이지만, AISstream.io처럼 바이너리/텍스트 혼합 서버에는 부적합하다.

솔루션

TextWebSocketHandler → AbstractWebSocketHandler 로 변경하고, 텍스트/바이너리 핸들러를 모두 구현.

```
public class AisStreamWebSocketClient extends
    AbstractWebSocketHandler {

    @Override
    protected void handleTextMessage(WebSocketSession session,
        TextMessage message) {
        // 구독 확인, 에러 메시지 등 텍스트 프레임 처리
        String payload = message.getPayload();
        processAisMessage(payload);
    }

    @Override
    protected void handleBinaryMessage(WebSocketSession session,
        BinaryMessage message) {
        // AIS 위치 데이터 - 바이너리 프레임을 UTF-8 디코딩
        byte[] bytes = message.getPayload().array();
        String payload = new String(bytes, StandardCharsets.UTF_8);
        processAisMessage(payload);
    }

    private void processAisMessage(String payload) {
        try {
            AisStreamResponse response =
                objectMapper.readValue(payload, AisStreamResponse.class);
            if (response.getMessageType().equals("PositionReport")) {
                AisPositionMessage position = mapToPosition(response);
                kafkaTemplate.send("ais-positions",
                    String.valueOf(position.getMmsi()), position);
            }
        } catch (JsonProcessingException e) {
            log.warn("AIS 메시지 파싱 실패: {}", e.getMessage());
        }
    }

    @Override
    public void afterConnectionEstablished(WebSocketSession session)
        throws Exception {
        log.info("AISstream 연결 성공, 구독 메시지 전송");
        String subscribeMsg = objectMapper.writeValueAsString(
            new AisSubscribeRequest(aisStreamApiKey, boundingBoxes));
        session.sendMessage(new TextMessage(subscribeMsg));
    }
}
```

```
}  
  
}
```

결과 (Before / After)

지표	Before	After
WebSocket 핸들러	TextWebSocketHandler	AbstractWebSocketHandler
바이너리 프레임 처리	1003 Close 전송	UTF-8 디코딩 후 처리
연결 상태	연결→1003→재연결 무한 루프	안정적 장기 연결 유지
AIS 메시지 수신	0 msg/s	70+ msg/s
Kafka <code>ais-positions</code>	0건	5분에 5,000+건
지도 선박 표시	0척	5,000+척 (5분 누적)

OpenSanctions 수집기 SUCCESS + 0건: URL 오류 + DataBufferLimitException → 2,002건 수집

배경 및 문제정의

OpenSanctions 제재 데이터 수집기가 `CollectorStatus.SUCCESS` 를 반환하면서 Kafka로 0건의 제재 데이터를 발송했다. 두 단계에 걸쳐 문제가 발견되었다.

1단계: API URL(`index.json`)이 메타데이터만 반환하여 entities 배열이 없었음. 2단계: 올바른 URL(`entities.ftm.json`)로 변경 후 `DataBufferLimitException` 이 발생하여 응답 수신 자체가 실패.

기술 선정 (대안 비교)

대안	장점	단점	선택
RestTemplate + InputStream 스트리밍	메모리 효율적	블로킹 I/O, Reactive 파이프라인과 불일치	X
WebClient + 버퍼 크기 증가 (10MB)	간단한 설정 변경	대용량 시 메모리 부담	O
WebClient + Flux 스트리밍 (line-by-line)	메모리 최적, 백프레서	NDJSON 파싱 복잡도 증가	향후 개선
파일 다운로드 후 로컬 파싱	네트워크 재시도 용이	디스크 I/O 추가, 임시파일 관리	X

분석

초기 판단 (1단계 - 잘못된 URL)

OpenSanctions API 문서에서 `https://data.opensanctions.org/datasets/latest/default/index.json` 을 엔드포인트로 사용. 응답은 데이터셋 메타데이터(이름, 설명, 통계)만 포함. `entities` 키가 존재하지 않으므로 빈 리스트로 처리 → 0건.

```
// index.json 응답 (메타데이터)
{
  "name": "default",
  "title": "OpenSanctions Default",
  "resources": [
    { "name": "entities.ftm.json", "url": "https://data.opensanctions.org/.../entities.ftm.json" }
  ]
}
```

```
}  
}
```

실제 데이터는 `resources[]`에 명시된 `entities.ftm.json` (NDJSON 형식, 약 180MB).

실제 원인 (2단계 - 버퍼 초과)

URL 수정 후 WebClient로 요청하면 Spring WebFlux의 기본 버퍼 제한(256KB)을 초과하여

`DataBufferLimitException: Exceeded limit on max bytes to buffer: 262144` 발생.

CS 원리: NDJSON, Reactive Streams, DataBuffer 메모리 관리

NDJSON vs JSON Array: 일반 JSON Array(`[{...}, {...}]`)는 전체를 파싱해야 구조를 알 수 있다. NDJSON(Newline-Delimited JSON)은 각 줄이 독립적인 JSON 객체로, 줄 단위 스트리밍 파싱이 가능하다.

```
// JSON Array - 전체 버퍼링 필수  
[{"id":"Q123","schema":"Person","properties":{"name":["Kim"]}},  
 {"id":"Q456","schema":"Person","properties":{"name":["Lee"]}}]  
  
// NDJSON - 줄 단위 파싱 가능  
{"id":"Q123","schema":"Person","properties":{"name":["Kim"]}}  
{"id":"Q456","schema":"Person","properties":{"name":["Lee"]}}
```

Reactive Streams Backpressure: Reactor의 `Flux` 는 Publisher-Subscriber 모델에서

Subscriber가 처리 가능한 만큼만 요청하는 **backpressure** 메커니즘을 제공한다.

`WebClient.retrieve().bodyToMono(String.class)` 는 전체 응답을 하나의 `String` 으로 버퍼링하므로 backpressure의 이점을 상실한다.

Spring WebFlux DataBuffer 메모리 관리: Netty 기반 WebClient는 `ByteBuf` 풀

(`PooledByteBufAllocator`)을 사용하여 Direct Memory에서 버퍼를 할당한다. `DataBuffer` 는 이 `ByteBuf` 의 래퍼로, reference counting으로 메모리를 관리한다.

```
HTTP 응답 수신 흐름:  
Network → Netty Channel → ByteBuf (Direct Memory) → DataBuffer → Decoder  
  
기본 제한: maxInMemorySize = 256KB (256 * 1024 = 262,144 bytes)  
이 제한은 Decoder(Jackson, StringDecoder 등)가 전체 응답을 메모리에 축적할 때 적용.
```

제한이 존재하는 이유는 **메모리 고갈 방지**다. 악의적이거나 예상치 못한 대용량 응답이 JVM Direct Memory를 소진하면 `OutOfDirectMemoryError` 가 발생하여 전체 애플리케이션이 중단된다.

ExchangeStrategies codec 설정: `ExchangeStrategies` 는 WebClient의 인코더/디코더 설정을 관리한다. `maxInMemorySize` 를 변경하면 내부적으로 `Jackson2JsonDecoder` , `StringDecoder` 등 모든 codec의 버퍼 제한이 조정된다.

스트리밍 vs 버퍼링 트레이드오프: 버퍼링(`bodyToMono`)은 구현이 간단하지만 메모리 사용량이 응답 크기에 비례. 스트리밍(`bodyToFlux` + `line split`)은 일정한 메모리로 무한 크기 응답 처리 가능하지만 에러 처리와 재시도 로직이 복잡해진다.

솔루션

```
@Configuration
public class WebClientConfig {

    @Bean
    public WebClient sanctionsWebClient() {
        ExchangeStrategies strategies = ExchangeStrategies.builder()
            .codecs(configurer -> configurer
                .defaultCodecs()
                .maxInMemorySize(10 * 1024 * 1024)) // 10MB
            .build();

        return WebClient.builder()
            .exchangeStrategies(strategies)
            .build();
    }
}
```

```
@RequiredArgsConstructor
public class OpenSanctionsCollector implements SanctionDataCollector {

    private final WebClient sanctionsWebClient;
    private final ObjectMapper objectMapper;
    private final KafkaTemplate<String, SanctionEvent> kafkaTemplate;

    private static final String ENTITIES_URL =
        "https://data.opensanctions.org/datasets/latest/default/entities.ftm.json";

    @Override
    public CollectorResult collect() {
        String ndjsonBody = sanctionsWebClient.get()
            .uri(ENTITIES_URL)
            .retrieve()
            .bodyToMono(String.class)
            .block(Duration.ofMinutes(3));

        if (ndjsonBody == null || ndjsonBody.isBlank()) {
            return CollectorResult.failure("Empty response from OpenSanctions");
        }

        // NDJSON 줄 단위 파싱
        List<SanctionEntity> entities = ndjsonBody.lines()
```

```

        .filter(line -> !line.isBlank())
        .map(this::parseEntity)
        .filter(Objects::nonNull)
        .filter(entity -> "LegalEntity".equals(entity.getSchema())
            || "Person".equals(entity.getSchema()))
        .toList();

    entities.forEach(entity ->
        kafkaTemplate.send("sanctions-data",
            entity.getId(), SanctionEvent.from(entity)));

    return CollectorResult.success(entities.size());
}

private SanctionEntity parseEntity(String line) {
    try {
        return objectMapper.readValue(line, SanctionEntity.class);
    } catch (JsonProcessingException e) {
        log.debug("NDJSON 라인 파싱 스킵: {}", e.getMessage());
        return null;
    }
}
}

```

결과 (Before / After)

지표	Before (1단계)	Before (2단계)	After
API URL	index.json (메타데이터)	entities.ftm.json	entities.ftm.json
WebClient 버퍼	256KB (기본값)	256KB (기본값)	10MB
에러	없음 (빈 응답 정상 처리)	DataBufferLimitException	없음
CollectorStatus	SUCCESS	FAILURE	SUCCESS
제재 데이터 수집	0건	0건	2,002건
파싱 형식	JSON Array 시도	N/A (수신 실패)	NDJSON line-by-line

AIS Vessel 자동 생성 미구현 → Neo4j 0척에서 7,573척, 지도 표시 5,266척 달성

배경 및 문제정의

- **상황**: 데모 데이터 25척 삭제 후 Neo4j Vessel 노드 0개. AIS 스트림으로 수만 건의 선박 위치 데이터가 Kafka를 통해 유입되지만, 지도에 선박이 단 한 척도 표시되지 않음
- **문제 1 (핵심)**: `AisNeo4jConsumer` 가 `vesselRepository.findByMmsi(mmsi).ifPresent(...)` 패턴 사용 — 기존 Vessel만 업데이트하고, Neo4j에 없는 MMSI는 전부 스킵. 새 Vessel 생성 로직 부재
- **문제 2 (IMO 검증)**: AIS 데이터의 대부분 선박은 IMO가 빈 문자열. `Imo.of("")` 호출 시 7자리 검증 실패로 `IllegalArgumentException` 발생하여 Consumer 전체가 중단
- **문제 3 (상태 전환)**: `Vessel.changeStatus()` 가 도메인 불변식으로 상태 전환을 검증 (예: UNKNOWN → FISHING_ENGAGED 거부). AIS 원시 데이터는 어떤 상태든 올 수 있어 정상 데이터가 거부됨

기술 선정 (대안 비교 테이블)

항목	Vessel 자동 생성 + nullable VO	별도 AisVessel 엔티티 분리	ACL에서 사전 필터링
구현 복잡도	낮음 (기존 모델 확장)	높음 (새 엔티티 + 매핑)	중간
데이터 정합성	VO null 허용으로 유연	완전 분리로 안전	누락 위험
도메인 모델 영향	최소 (메서드 추가)	없음 (별도 모델)	없음
선택 이유	채택 — 기존 온톨로지 활용	중복 모델 관리 부담	AIS 데이터 손실

분석

- 초기 판단: AIS Kafka Consumer의 메시지 수신 자체가 안 되는 것으로 의심 → 로그 확인 결과 메시지는 정상 수신되나 `ifPresent` 분기에서 전부 스킵
- 실제 원인: DDD Aggregate Root(Vessel)의 불변식이 외부 시스템(AIS) 데이터의 특성과 충돌. 세 가지 방어 로직이 동시에 데이터 유입을 차단:
 1. `findByMmsi().ifPresent()` — 존재하지 않으면 아무것도 안 함
 2. `Imo` Value Object — 빈 문자열을 유효하지 않은 IMO로 거부
 3. `changeStatus()` — 도메인이 허용하지 않는 상태 전환 거부
- CS 원리:
 - **DDD Conformist 패턴**: AIS는 외부 시스템(Upstream)이므로 내부 도메인이 외부 데이터 형식에 맞춰야 함. Aggregate Root 불변식을 외부 이벤트에 그대로 적용하면 데이터 유입이 차단됨
 - **Value Object nullable 설계**: 필수 VO(MMSI — 식별자)와 선택 VO(IMO, Flag — 보강 데이터)를 구분해야 함. AIS 데이터에서 IMO는 선택 필드
 - **External Event vs Domain Command**: Event Sourcing 관점에서 AIS 데이터는 외부 이벤트(External Event). 내부 도메인 규칙(상태 전환 검증)은 도메인 커맨드에만 적용해야 하며, 외부 이벤트는 있는 그대로 수용해야 함
 - **CQRS Write/Read 불일치**: Write Model(도메인 규칙 엄격)과 Read Model(지도 표시 용 유연) 간의 요구사항 차이. AIS 데이터 수집은 Read Model 쪽에 가까움

솔루션

1) AisNeo4jConsumer — findByMmsi 없으면 새 Vessel 생성

```
// Before: 기존 Vessel만 업데이트
vesselRepository.findByMmsi(mmsi).ifPresent(vessel -> {
    vessel.updatePosition(lat, lon);
});

// After: 없으면 자동 생성
Vessel vessel = vesselRepository.findByMmsi(mmsi)
    .orElseGet(() -> vesselRepository.save(
        Vessel.builder()
            .mmsi(Mmsi.of(mmsi))
            .imo(imo.isBlank() ? null : Imo.of(imo))           // nullable
            .flag(flag.isBlank() ? null : Flag.of(flag))       //
            nullable
            .name(shipName)
            .vesselType(VesselType.from(shipType))
            .status(NavigationStatus.fromAisCode(navStatus))
            .build()
    ));
vessel.updatePositionFromAis(lat, lon, heading, speed);
```

2) Vessel 도메인 — 외부 이벤트용 상태 업데이트 메서드 추가


```
// 기존: 도메인 커맨드용 (상태 전환 검증)
public void changeStatus(NavigationStatus newStatus) {
    if (!this.status.canTransitionTo(newStatus)) {
        throw new InvalidStatusTransitionException(this.status,
            newStatus);
    }
    this.status = newStatus;
}

// 추가: 외부 이벤트용 (AIS 데이터 - 검증 없이 수용)
public void updateStatusFromAis(NavigationStatus aisStatus) {
    this.status = aisStatus; // External Event는 있는 그대로 반영
}
```

3) Neo4jNodeMapper — null-safe 매핑

```
// IMO, Flag가 null일 수 있으므로 매핑 시 null 체크
.imo(node.get("imo").isNull() ? null :
    Imo.of(node.get("imo").asString()))
.flag(node.get("flag").isNull() ? null :
    Flag.of(node.get("flag").asString()))
```

결과 (Before/After 수치 테이블)

지표	Before	After
Neo4j Vessel 노드	0개	7,573개
Map API 응답 선박 수	0척	5,266척 (Redis GEO 매칭분)
AIS Consumer 에러율	IMO 검증 실패로 다수 중단	0% (null-safe 처리)
상태 전환 거부	정상 AIS 데이터 거부	외부 이벤트 전량 수용
선박 상세 정보 완성도	N/A	IMO 보유 약 30%, Flag 보유 약 60%